SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Analytical Problem Solving)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO3-AT1-ANALYTICAL PROBLEM SOLVING (ABQ)
Question	Q2 — Debugging and Rewriting: Simple Banking System (10 Marks)

Q2. (Debugging and Rewriting) [10 Marks]

The program below simulates a simple banking system using lists and dictionaries. It has FOUR bugs. Identify each bug with reason and rewrite the correct program.

Original (Buggy) Program

```
accounts = {}
def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}
def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] =+ amount
        accounts[acc_no]['txns'].append(('Deposit', amount))
def withdraw(acc_no, amount):
    if accounts[acc_no]['balance'] > amount:
        accounts[acc_no]['balance'] -= amount
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')
def mini_statement(acc_no):
    for t in accounts[acc_no]['txns'][-5]:
        print(t)
create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

Bug Report

Bug 1 — Wrong operator in deposit(): `accounts[acc_no]['balance'] =+ amount` uses `=+` (assignment of unary plus), which is the same as `= amount`. This overwrites the balance with the deposit amount instead of adding to it. **Fix:** use the compound operator `+= amount`.

Bug 2 — No existence check in withdraw(): unlike deposit(), withdraw() accesses `accounts[acc_no]` directly without checking `if acc_no in accounts`. Calling withdraw() with an account number that was never created raises a KeyError. **Fix:** add the same existence check used in deposit(), and print a message such as 'Account not found' when the account doesn't exist.

Bug 3 — Incorrect boundary condition in withdraw(): `if accounts[acc_no]['balance'] > amount:` uses strict greater-than, which wrongly blocks a withdrawal that exactly matches the available balance (e.g. balance = 3000, withdraw 3000 should succeed). **Fix:** use >= so a withdrawal equal to the

balance is allowed.

Bug 4 — Indexing instead of slicing in mini_statement(): `accounts[acc_no]['txns'][-5]` retrieves a single transaction (the 5th-from-last item) rather than the last five transactions, and raises an IndexError whenever there are fewer than 5 transactions. **Fix:** use the slice `[-5:]` to safely get up to the last five transactions.

Corrected Program

```
"""
CO3-AT1 - Analytical Problem Solving (ABQ)
Q2: Debugging and Rewriting - Simple Banking System [10 Marks]
------------------------------------------------------------
The corrected program below simulates a simple banking system
using lists and dictionaries. All four bugs from the original
version have been identified and fixed (see Bug Report).
"""

accounts = {}


def create_account(acc_no, name, balance):
    """Creates a new account record with an empty transaction list."""
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}


def deposit(acc_no, amount):
    """Adds the deposited amount to the account balance and logs it."""
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount          # Bug 1 fix: += not =+
        accounts[acc_no]['txns'].append(('Deposit', amount))


def withdraw(acc_no, amount):
    """Withdraws the amount if the account exists and funds are sufficient."""
    if acc_no in accounts:                              # Bug 2 fix: existence check added
        if accounts[acc_no]['balance'] >= amount:        # Bug 3 fix: >= not >
            accounts[acc_no]['balance'] -= amount
            accounts[acc_no]['txns'].append(('Withdraw', amount))
        else:
            print('Insufficient balance')
    else:
        print('Account not found')


def mini_statement(acc_no):
    """Prints the last five transactions for the given account."""
    for t in accounts[acc_no]['txns'][-5:]:              # Bug 4 fix: [-5:] slice not [-5] index
        print(t)


create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

Sample Output

```
Insufficient balance
('Deposit', 5000)
('Withdraw', 3000)
Balance: 12000
```

Conclusion

The original program failed because of a mistyped assignment operator, a missing account-existence check, an off-by-one boundary condition, and a list index used where a slice was required. After correcting all four issues, running the program on the given calls produces the expected trace: the 20000 withdrawal correctly reports insufficient balance, the mini statement lists the deposit and the successful 3000 withdrawal, and the final balance is correctly reported as 12000 (10000 opening balance + 5000 deposit – 3000 withdrawal).